

Day 12 – ML Fundamentals, Math, Classical ML, NLP & CV

1. The Big Picture

Machine learning is the process of learning patterns from data so a system can make predictions, classifications, recommendations, or decisions.

A typical ML lifecycle is:

Data → Features → Model training → Validation → Testing → Deployment → Monitoring

In modern GenAI systems, classical ML still appears around the LLM:

- Intent classification before sending a query to an LLM
 - Spam, toxicity, fraud, or policy detection
 - Document ranking and reranking in RAG
 - Predicting model latency, cost, or answer quality
 - Routing requests to different models
 - Computer vision models for image understanding
 - Reinforcement learning for aligning language models
-

2. ML Basics

Supervised Learning

In supervised learning, every training example has:

- **Input:** the information given to the model
- **Label:** the correct answer the model should learn to predict

Example:

Email text → Spam or Not Spam
House features → House price
Customer query → Intent category

There are two main supervised-learning tasks.

Classification

Predict a category.

Examples:

- Fraud or not fraud
- Positive, neutral, or negative sentiment
- Billing, technical support, or account-related query

Regression

Predict a continuous number.

Examples:

- House price
- Delivery time

- LLM response latency
- Expected token usage

GenAI example

Before processing a user request, a small classifier might predict:

Query → Simple question / RAG needed / Tool call needed / Unsafe request

This classifier may be faster and cheaper than using an LLM for every routing decision.

Unsupervised Learning

Unsupervised learning works with data that does not have explicit labels.

The model attempts to discover patterns or structure.

Common tasks include:

- Clustering similar items
- Finding unusual behavior
- Reducing the number of dimensions
- Discovering hidden groups in data

Example:

Thousands of support tickets
↓
Automatically group them into:
– Payment problems
– Login problems
– Delivery problems

GenAI example

Embedding vectors from customer queries can be clustered to discover:

- Common user intents
 - Missing documentation topics
 - Repeated failure patterns
 - New categories not present in the existing taxonomy
-

Reinforcement Learning

In reinforcement learning, an **agent** interacts with an environment.

The agent:

1. Observes the current state
2. Chooses an action
3. Receives a reward or penalty
4. Learns which actions produce better long-term outcomes

State → Action → Reward → Learning

Examples:

- Game-playing agents
- Robot navigation
- Resource allocation
- Sequential decision-making

GenAI example

Large language models can be aligned using human or automated feedback.

A simplified RLHF flow is:

```
Prompt
↓
Model generates multiple responses
↓
Humans rank the responses
↓
Reward model learns human preferences
↓
LLM is optimized toward higher-reward responses
```

In interviews, avoid saying that every AI agent uses reinforcement learning. Most LLM agents use prompting, tools, state machines, or workflow orchestration rather than online RL.

3. Train, Validation, and Test Splits

Data is commonly divided into three parts.

Training set

Used to learn model parameters.

Example:

```
The model adjusts its weights using training examples.
```

Validation set

Used during development to:

- Select hyperparameters
- Compare models
- Choose thresholds
- Decide when to stop training
- Detect overfitting

Hyperparameters are configuration choices such as:

- Learning rate
- Tree depth
- Regularization strength
- Number of training epochs

Test set

Used only after model development to estimate performance on unseen data.

Training data	→ Learn parameters
Validation data	→ Make development decisions
Test data	→ Final unbiased evaluation

A common split is:

70% training
15% validation
15% testing

The exact percentages depend on dataset size.

Important interview issue: data leakage

Data leakage happens when information from validation or test data influences training.

Examples:

- Normalizing the entire dataset before splitting
- Including future information in a time-series model
- Having nearly identical documents in training and testing
- Selecting prompts based repeatedly on test-set results

Leakage produces unrealistically strong evaluation results.

For time-dependent data, use a chronological split:

Older data	→ Training
Recent data	→ Validation
Newest data	→ Testing

4. Overfitting and Regularization

Overfitting

Overfitting happens when a model learns the training data too closely, including noise and accidental patterns.

Symptoms:

Training performance: Excellent
Validation performance: Poor

The model memorizes rather than generalizes.

Underfitting

Underfitting happens when the model is too simple or insufficiently trained.

Symptoms:

Training performance: Poor
Validation performance: Poor

Ways to reduce overfitting

- Collect more data
- Use a simpler model
- Remove unnecessary features
- Apply regularization
- Use dropout
- Stop training earlier
- Use data augmentation
- Perform cross-validation

L2 Regularization

L2 regularization discourages the model from using extremely large weights.

The training objective becomes approximately:

Total loss = Prediction loss + $\lambda \times$ Sum of squared weights

Where λ controls regularization strength.

Intuition:

| The model is rewarded for fitting the data but penalized for becoming unnecessarily complex.

A large λ produces stronger regularization. Too much regularization can cause underfitting.

Dropout

Dropout is mainly used in neural networks.

During training, some neurons are randomly disabled.

Normal network:

A → B → C → D

With dropout during one training step:

A → X → C → D

This prevents neurons from depending too strongly on one another and encourages the network to learn more robust patterns.

Dropout is generally active during training and disabled during normal inference.

5. Evaluation Metrics

The correct metric depends on the business problem.

Confusion matrix

For binary classification:

Actual / Predicted	Positive	Negative
Positive	True Positive	False Negative
Negative	False Positive	True Negative

Example for fraud detection:

- **True positive:** Fraud correctly detected
- **False positive:** Legitimate transaction marked as fraud
- **False negative:** Fraud incorrectly allowed
- **True negative:** Legitimate transaction correctly allowed

Accuracy

Accuracy = Correct predictions / Total predictions

Accuracy works well when classes are reasonably balanced and mistakes have similar costs.

Problem with imbalanced data

Suppose only 1% of transactions are fraudulent.

A model that always predicts “not fraud” obtains 99% accuracy but detects no fraud.

Therefore, accuracy alone can be misleading.

Precision

Precision = $TP / (TP + FP)$

Precision answers:

| Of everything predicted as positive, how many were actually positive?

Use precision when false positives are costly.

Examples:

- Automatically blocking user accounts
- Flagging content for removal
- Sending expensive cases for human review

High precision means the positive predictions are trustworthy.

Recall

Recall = $TP / (TP + FN)$

Recall answers:

Of all actual positive cases, how many did the model find?

Use recall when missing a positive case is costly.

Examples:

- Disease screening
- Fraud detection
- Security-threat detection
- Retrieving relevant documents for RAG

In first-stage RAG retrieval, high recall is usually important because a relevant document that is never retrieved cannot be recovered by the reranker or LLM.

F1 Score

$$F1 = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

F1 balances precision and recall.

It is useful when:

- Classes are imbalanced
- Both false positives and false negatives matter
- You want one summary metric

F1 does not include true negatives directly.

ROC-AUC

ROC-AUC measures how well a classifier ranks positive examples above negative examples across many decision thresholds.

- 1.0: excellent separation
- 0.5: approximately random ranking

A useful interpretation:

If one positive and one negative example are selected, ROC-AUC reflects the probability that the classifier ranks the positive example higher.

For severely imbalanced datasets, also inspect precision-recall metrics because ROC-AUC can sometimes look strong despite poor positive-class performance.

Classification metric selection

Scenario	Important metric
Balanced classes, similar mistake costs	Accuracy
False positives are expensive	Precision
False negatives are expensive	Recall
Precision and recall both matter	F1

Compare ranking across thresholds	ROC-AUC
-----------------------------------	---------

6. Regression Metrics

Mean Squared Error

$$\text{MSE} = \text{Average of } (\text{actual} - \text{predicted})^2$$

Because errors are squared, large errors receive a much larger penalty.

Example:

$$\begin{aligned} \text{Error} = 2 &\rightarrow \text{Squared error} = 4 \\ \text{Error} = 10 &\rightarrow \text{Squared error} = 100 \end{aligned}$$

Use MSE when:

- Large errors are especially undesirable
- Smooth mathematical optimization is useful
- Outliers are meaningful and should be penalized strongly

Disadvantage: it is sensitive to outliers.

Mean Absolute Error

$$\text{MAE} = \text{Average of } |\text{actual} - \text{predicted}|$$

MAE treats errors more linearly.

Use MAE when:

- You want an interpretable average error
- The dataset contains outliers
- A few large errors should not dominate the metric

If the MAE for latency prediction is 120 ms, predictions are wrong by approximately 120 ms on average.

MSE versus MAE

Property	MSE	MAE
Large-error penalty	Strong	Linear
Outlier sensitivity	High	Lower
Units	Squared units	Original units
Optimization	Smooth	Less smooth at zero

7. Classical ML Algorithms

Linear Regression

Linear regression predicts a continuous value using a weighted combination of features.

For one feature:

$$\hat{y} = wx + b$$

For multiple features:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Example:

```
LLM latency =  
  weight1 × input tokens  
+ weight2 × output tokens  
+ weight3 × model size  
+ bias
```

Strengths

- Simple
- Fast
- Interpretable
- Useful as a baseline

Limitations

- Assumes approximately linear relationships
- Sensitive to outliers
- May underfit complex data

Logistic Regression

Despite its name, logistic regression is primarily a classification algorithm.

It first computes a weighted score:

$$z = w \cdot x + b$$

It then converts that score into a probability using the sigmoid function:

$$\text{Probability} = 1 / (1 + e^{-z})$$

Example:

$$P(\text{query is unsafe}) = 0.91$$

The system then applies a threshold:

If probability ≥ 0.8 :
classify as unsafe

Changing the threshold changes the precision-recall trade-off.

Good use cases

- Binary classification
- Interpretable baselines
- High-dimensional sparse text features
- Fast intent or risk classifiers

Decision Trees

A decision tree repeatedly splits data using rules.

```
Is query length > 500?  
├── Yes: Does it contain an attachment?  
│   ├── Yes → Large-context model  
│   └── No  → Standard model  
└── No  → Small model
```

Strengths

- Easy to explain
- Handles nonlinear relationships
- Supports numerical and categorical features
- Requires less feature scaling

Limitations

- Can overfit easily
- Small data changes may create a different tree
- A single tree may be unstable

Tree depth, minimum samples per leaf, and pruning help control overfitting.

Random Forest

A random forest combines many decision trees.

Each tree is trained using:

- A random sample of training records
- A random subset of features

The outputs are combined through:

- Majority vote for classification
- Average prediction for regression

Intuition:

One tree may make an unstable decision, but many diverse trees voting together are usually more reliable.

Strengths

- Strong classical baseline
- Captures nonlinear relationships
- Less overfitting than a single tree
- Works well with tabular data

Limitations

- Less interpretable than one tree
- Larger and slower than one tree
- Usually not the natural choice for raw text, images, or very high-dimensional embeddings

GenAI example

A random forest might predict whether an LLM request will:

- Exceed a latency target
 - Require escalation
 - Produce a low-quality response
 - Need a larger model
-

8. Math for Machine Learning

Scalars, vectors, and matrices

Scalar

A single number:

```
Learning rate = 0.001
```

Vector

An ordered list of numbers:

```
User embedding = [0.21, -0.14, 0.72, ...]
```

A vector can represent:

- A document
- A query
- An image
- Model parameters
- Feature values

Matrix

A two-dimensional arrangement of numbers:

```
[  
  [1, 2, 3],  
  [4, 5, 6]  
]
```

Matrices can represent:

- A batch of vectors
 - Neural-network weights
 - Token embeddings for a sequence
 - Image pixel values
 - Feature tables
-

Dot Product

For vectors:

$$\begin{aligned} \mathbf{a} &= [a_1, a_2, \dots, a_n] \\ \mathbf{b} &= [b_1, b_2, \dots, b_n] \\ \mathbf{a} \cdot \mathbf{b} &= a_1b_1 + a_2b_2 + \dots + a_nb_n \end{aligned}$$

Example:

$$\begin{aligned} [1, 2] \cdot [3, 4] \\ &= 1 \times 3 + 2 \times 4 \\ &= 11 \end{aligned}$$

Intuitively, the dot product measures how strongly two vectors align, although its magnitude is also affected by vector lengths.

Dot products are heavily used in:

- Neural-network layers
 - Attention mechanisms
 - Embedding similarity
 - Matrix multiplication
-

Cosine Similarity

Cosine similarity measures the angle between two vectors:

$$\text{cosine_similarity}(\mathbf{a}, \mathbf{b}) = (\mathbf{a} \cdot \mathbf{b}) / (||\mathbf{a}|| \times ||\mathbf{b}||)$$

Typical interpretation:

- Near 1: similar direction
- Near 0: unrelated or orthogonal
- Near -1: opposite direction

RAG example

```
User query:
"How do I reset my password?"

Document A:
"Steps to change or reset your password"
```

Document B:
"Company holiday calendar"

The query embedding should have higher cosine similarity with Document A.

Cosine similarity focuses on direction rather than raw vector magnitude.

9. Gradients and Backpropagation

Loss function

A loss function measures how wrong a model is.

Examples:

- MSE for regression
- Cross-entropy for classification
- Token prediction loss for language models

Training aims to reduce loss.

Gradient

A gradient tells us:

- Which direction changes the loss
- How strongly each parameter affects the loss

Imagine standing on a hill in fog. The gradient indicates the steepest uphill direction. To move downhill, move in the opposite direction.

$$\text{new_weight} = \text{old_weight} - \text{learning_rate} \times \text{gradient}$$

The learning rate controls step size.

- Too large: training may overshoot or become unstable
- Too small: training may be extremely slow

Backpropagation

A neural network contains many connected mathematical operations.

Backpropagation calculates how much each parameter contributed to the final error by applying the chain rule backward through the network.

Forward pass:
Input → Predictions → Loss

Backward pass:
Loss → Gradients for each layer

Update:
Weights move in the direction that reduces loss

A common interview explanation is:

Backpropagation efficiently computes gradients from the output layer back through all earlier layers. An optimizer then uses those gradients to update the parameters.

Backpropagation computes gradients; gradient descent or another optimizer performs the update.

10. Probability Fundamentals

Random Variable

A random variable assigns a numerical value to the outcome of a random process.

Examples:

X = Number of failed requests in one minute
Y = LLM response latency
Z = Whether a user clicks a recommendation

A discrete random variable has countable values. A continuous random variable can take values over a range.

Expectation

Expectation is the probability-weighted average value.

$$E[X] = \sum x \times P(X = x)$$

Intuition:

What value should we expect on average over many repetitions?

Examples:

- Expected inference cost per request
 - Expected number of tool calls
 - Expected response latency
-

Variance

Variance measures how spread out values are around the mean.

$$\text{Var}(X) = E[(X - E[X])^2]$$

Two services may both average 500 ms, but one may consistently respond near 500 ms, while another varies between 100 ms and 2 seconds.

The second service has higher variance and may be less predictable.

Conditional Probability

Conditional probability represents the probability of event A given that B has happened.

$$P(A \mid B)$$

Example:

$$P(\text{answer is incorrect} \mid \text{retrieval returned no relevant document})$$

This is different from:

$$P(\text{retrieval returned no relevant document} \mid \text{answer is incorrect})$$

The direction matters.

Bayes' Rule

$$P(A \mid B) = \frac{P(B \mid A) \times P(A)}{P(B)}$$

Bayes' rule updates prior beliefs after observing evidence.

Example:

- A: A message is spam
- B: The message contains a suspicious link

$$P(\text{spam} \mid \text{suspicious link})$$

depends on:

- How commonly spam contains suspicious links
- The overall rate of spam
- How common suspicious links are in all messages

ML relevance

Bayesian thinking appears in:

- Naive Bayes classifiers
 - Probabilistic reasoning
 - Uncertainty estimation
 - Updating beliefs from evidence
 - A/B testing and experimentation
-

11. NLP Basics

Tokenization

Language models do not directly process raw sentences. Text is converted into tokens.

Tokens may be:

- Whole words
- Parts of words
- Punctuation

- Special control symbols

Example:

```
"unbelievable"
```

might become:

```
["un", "believ", "able"]
```

Modern LLMs commonly use subword tokenization because it balances:

- Vocabulary size
- Handling rare words
- Handling new words
- Efficient text representation

Why tokenization matters

Tokenization affects:

- Context-window usage
- Inference cost
- Latency
- Chunk size in RAG
- Multilingual performance
- Maximum output size

A character count is not the same as a token count.

Word Embeddings

A word embedding represents a word as a dense numerical vector.

```
"king"    → [0.2, -0.7, 0.4, ...]  
"queen"   → [0.3, -0.6, 0.5, ...]  
"banana"  → [-0.8, 0.1, 0.2, ...]
```

Words appearing in similar contexts tend to receive similar vectors.

Word2Vec

Word2Vec learns embeddings from local word context.

Two common ideas are:

- **CBOW**: Predict a word from surrounding words
- **Skip-gram**: Predict surrounding words from a given word

Example:

```
"The cat sat on the mat"
```

The model learns that words such as cat, dog, and pet may appear in similar contexts.

GloVe

GloVe learns embeddings using global word co-occurrence statistics.

It considers how frequently words appear together across the full corpus.

High-level distinction:

Word2Vec	→	Learns mainly from local prediction tasks
GloVe	→	Uses global co-occurrence statistics

Both produce **static embeddings**: a word has one main vector regardless of context.

For example, bank would have the same basic vector in:

river bank
financial bank

Modern transformer models produce contextual embeddings, so the representation changes according to the sentence.

12. Computer Vision Basics

CNNs

A convolutional neural network, or CNN, is designed to process grid-like data such as images.

An image can be represented as:

Height × Width × Channels

For an RGB image, channels represent red, green, and blue.

CNNs apply small filters across the image to detect local patterns.

Early layers may detect:

- Edges
- Corners
- Simple textures

Middle layers may detect:

- Shapes
- Eyes
- Wheels
- Repeated patterns

Later layers may detect:

- Faces
- Cars
- Animals
- Complete objects

A simplified flow is:

```
Image
↓
Edges and textures
↓
Shapes and object parts
↓
Object-level representation
↓
Classification or detection
```

Why convolutions help

The same filter is reused across image locations. This is called weight sharing.

It reduces parameters and allows the model to detect a feature regardless of where it appears.

Transfer Learning

Training a large vision model from scratch requires substantial data and computing resources.

Transfer learning starts with a model already trained on a large dataset.

Typical process:

```
Pre-trained vision model
↓
Replace or modify final layer
↓
Train on smaller domain-specific dataset
```

Two common approaches:

Feature extraction

Freeze most pre-trained layers and train only a new output layer.

Use when:

- Dataset is small
- New task resembles the original training task
- Training resources are limited

Fine-tuning

Unfreeze some or all pre-trained layers and continue training with a small learning rate.

Use when:

- More domain-specific adaptation is required
- Sufficient labeled data is available

GenAI examples

Transfer learning is used for:

- Document image classification
- Defect detection
- Medical image analysis

- Image moderation
- Visual question answering
- Multimodal assistants
- OCR and document understanding

Modern multimodal systems often combine:

Vision encoder → Image representation → Language model → Text response

13. How These Concepts Fit into a GenAI Product

Consider an enterprise support assistant.

```

User question
  ↓
Intent and safety classifier
  ↓
Tokenization
  ↓
Query embedding
  ↓
Vector retrieval using cosine similarity
  ↓
Reranking
  ↓
LLM generation
  ↓
Quality and policy checks
  ↓
Response
  
```

The underlying concepts include:

Component	Relevant concept
Intent detection	Logistic regression, trees, transformer classifier
Query representation	Vectors and embeddings
Document retrieval	Dot product or cosine similarity
Reranking evaluation	Precision, recall, F1
LLM training	Gradients and backpropagation
Safety classification	Supervised learning
Query clustering	Unsupervised learning
Preference alignment	Reinforcement learning
Image understanding	CNNs or vision transformers
Latency prediction	Linear regression or random forest
Confidence reasoning	Probability and conditional probability

14. Common Interview Mistakes

1. **Using accuracy for every classification problem.** Always consider class imbalance and the costs of false positives and false negatives.
 2. **Treating validation and test data as interchangeable.** Validation supports development decisions; testing is for final unbiased evaluation.
 3. **Saying cosine similarity measures distance.** It measures angular similarity. Cosine distance can be derived from it.
 4. **Confusing logistic regression with regression tasks.** Logistic regression is usually used for classification.
 5. **Saying dropout is active during normal inference.** Standard dropout is normally used during training and disabled at inference.
 6. **Equating all LLM agents with reinforcement learning.** Most production agents use orchestration and tool calling without online RL.
 7. **Describing backpropagation as the optimizer.** Backpropagation computes gradients; the optimizer uses those gradients to update weights.
 8. **Evaluating only the model and ignoring the system.** GenAI systems also require latency, cost, retrieval quality, safety, and end-to-end task-success metrics.
-

15. Interview Q&A

1. What is the difference between supervised and unsupervised learning?

Supervised learning uses labeled input-output examples to learn predictions. Unsupervised learning uses unlabeled data to discover patterns such as clusters or latent structure.

2. Why do we need separate validation and test sets?

The validation set is used for model selection, threshold tuning, and hyperparameter decisions. The test set is reserved for estimating final performance on unseen data. Repeatedly tuning against the test set makes it part of the development process and biases the result.

3. How do you identify overfitting?

Overfitting usually appears when training performance keeps improving while validation performance stops improving or becomes worse. It can be reduced with regularization, simpler models, more data, dropout, early stopping, or augmentation.

4. When would you prioritize precision over recall?

Prioritize precision when false positives are especially costly. For example, automatically suspending legitimate accounts requires high precision because an incorrect positive prediction harms real users.

5. Why is recall important in a RAG system?

Recall measures how many relevant documents were retrieved. If the retriever misses the necessary document, the reranker and LLM never receive it, so the final answer may be incorrect even if the generator is strong.

6. What is the difference between linear and logistic regression?

Linear regression predicts continuous numerical values. Logistic regression predicts class probabilities, commonly using a sigmoid function for binary classification.

7. Why use cosine similarity for embeddings?

Cosine similarity compares vector direction while reducing the influence of vector magnitude. Embeddings with similar semantic meaning often point in similar directions, making cosine similarity useful for semantic retrieval.

8. Explain gradient descent and backpropagation simply.

Backpropagation computes how much each model parameter contributed to the error. Gradient descent uses those gradients to adjust parameters in the direction that reduces the loss.

9. How do Word2Vec and contextual embeddings differ?

Word2Vec produces one static vector per word. Contextual models produce different vectors based on surrounding text, allowing a word such as bank to have different representations in financial and river-related sentences.

10. What is transfer learning, and why is it useful?

Transfer learning starts with a model trained on a large dataset and adapts it to a smaller domain-specific task. It reduces training time, data requirements, and computation while often improving performance.

Final Interview Mental Model

Remember these five connections:

Labels determine the learning type. Business costs determine the metric. Vectors represent data. Gradients teach the model. Pre-trained models reduce data and compute requirements.
--

For GenAI interviews, connect traditional ML concepts to the complete production system—not only the LLM. Retrieval, routing, classification, safety, evaluation, vision processing, latency prediction, and monitoring all depend on these fundamentals.